

实验三 首次适应内存分配算法的实现

一、实验目的

1. 理解首次适应算法：通过实现首次适应（First Fit）内存分配算法，深入理解其工作原理和特性。
2. 掌握 Rust 编程：通过动手编写 Rust 代码，熟悉 Rust 语言的内存管理、指针操作、并发控制等特性。
3. 实践内存管理：通过实现自定义的内存分配器，理解操作系统中的内存管理机制。

二、实验原理

2.1 内存分配算法：

- 首次适应算法：在空闲内存块链表中，从头开始查找第一个满足请求大小的空闲块，并将其分配给请求者。该算法简单高效，但可能导致内存碎片。
- 最佳适应算法：在空闲内存块链表中，查找大小最接近请求大小的空闲块进行分配。该算法可以减少内存碎片，但效率较低。
- 最差适应算法：在空闲内存块链表中，查找最大的空闲块进行分配。该算法可以减少内存碎片，但可能导致大块的内存被分割。

2.2 Rust 内存管理：

- Rust 通过所有权、借用检查和生命周期管理内存，避免了传统 C/C++ 中的内存泄漏和悬挂指针问题。
- Rust 提供了全局分配器接口 `GlobalAlloc`，允许用户实现自定义的内存分配策略。

2.3 并发控制：

- Rust 通过 `std::sync::Mutex`、`std::sync::RwLock` 等并发原语，提供了线程安全的并发控制机制。
- 在实现自定义内存分配器时，需要考虑并发访问的同步问题。

三、实验内容

1. 定义数据结构。
2. 实现内存分配和释放。
3. 实现全局分配器接口。

四、实验步骤

4.1 代码结构概览

- **ListNode**: 用于表示内存中的一个空闲区域。包含区域的大小（**size**）和指向下一个空闲区域的指针（**next**）。
- **FirstFitAllocator**: 任务执行器，管理任务队列和任务调度。实现了首次适配内存分配器的核心逻辑。使用一个 **ListNode** 类型的 **head** 作为空闲区域的链表头。
- **GlobalAlloc**: Rust 的全局分配器接口，用于将 **FirstFitAllocator** 集成到全局内存分配体系中。

4.2 核心代码说明

[src/allocator/first_fit.rs](#)

链表节点定义:

```
use crate::println;
use super::{align_up, Locked};
use alloc::alloc::{GlobalAlloc, Layout};
use core::{mem, ptr};

// 链表结点定义
struct ListNode {
    size: usize,
    next: Option<&'static mut ListNode>,
}

impl ListNode {
    const fn new(size: usize) -> Self {
        ListNode { size, next: None }
    }

    fn start_addr(&self) -> usize {
        self as *const Self as usize
    }

    fn end_addr(&self) -> usize {
        self.start_addr() + self.size
    }
}
```

- **ListNode** 表示一个空闲区域，包含大小和指向下一个节点的指针。

首次适应分配器初始化:

```

/// 首次适配分配器
pub struct FirstFitAllocator {
    head: ListNode,
}

impl FirstFitAllocator {
    /// 创建一个新的 FirstFitAllocator 实例
    pub const fn new() -> Self {
        Self {
            head: ListNode::new(0),
        }
    }

    /// 初始化堆区域
    pub fn init(&mut self, heap_start: usize, heap_size: usize) {
        let mut node = ListNode::new(heap_size);
        node.next = self.head.next.take();
        self.head.next = Some(unsafe { &mut *(heap_start as *mut
ListNode) });
        unsafe {
            ptr::write(heap_start as *mut ListNode, node);
        }
    }
}

```

- 初始化堆时，将整个堆空间作为一个初始空闲区域。

内存分配逻辑:

```

/// 查找合适的空闲区域
fn find_region(&mut self, size: usize, align: usize) ->
Option<(&'static mut ListNode, usize)> {
    let mut current = &mut self.head;

    while let Some(ref mut region) = current.next {
        if let Ok(alloc_start) = Self::alloc_from_region(region,
size, align) {
            let next = region.next.take();
            let ret = Some((current.next.take().unwrap(),
alloc_start));
            current.next = next;
            return ret;
        }
        current = current.next.as_mut().unwrap();
    }
    None
}

```

```

    }

    /// 从区域中分配内存
    fn alloc_from_region(region: &ListNode, size: usize, align: usize)
    -> Result<usize, ()> {
        let alloc_start = align_up(region.start_addr(), align);
        let alloc_end = alloc_start.checked_add(size).ok_or(())?;

        if alloc_end > region.end_addr() {
            return Err(());
        }

        Ok(alloc_start)
    }

```

- 遍历链表，找到第一个满足分配需求的空闲区域。

内存回收逻辑:

```

    /// 添加一个空闲区域
    fn add_free_region(&mut self, addr: usize, size: usize) {
        let mut current = &mut self.head;
        let new_node = ListNode::new(size);
        let new_node_addr = addr;
        let new_node_end = addr + size;

        // 寻找合适的位置插入新节点
        while let Some(ref mut next) = current.next {
            if next.start_addr() == new_node_end {
                // 可以与下一个节点合并
                let mut merged = ListNode::new(size + next.size);
                merged.next = next.next.take();
                unsafe {
                    (addr as *mut ListNode).write(merged);
                }
                current.next = Some(unsafe { &mut *(addr as *mut
ListNode) });
                return;
            } else if next.end_addr() == new_node_addr {
                // 可以与当前节点合并
                next.size += size;
                // 检查是否可以与下一个节点合并
                let next_end_addr = next.end_addr();
                if let Some(ref mut next_next) = next.next {
                    if next_end_addr == next_next.start_addr() {

```

```

        next.size += next_next.size;
        next.next = next_next.next.take();
    }
}
return;
} else if next.start_addr() > new_node_addr {
    // 在这里插入新节点
    let mut node = ListNode::new(size);
    node.next = current.next.take();
    unsafe {
        (addr as *mut ListNode).write(node);
    }
    current.next = Some(unsafe { &mut *(addr as *mut
ListNode) });
    return;
}
current = current.next.as_mut().unwrap();
}

// 如果没有找到合适的位置，添加到链表末尾
let mut node = ListNode::new(size);
node.next = None;
unsafe {
    (addr as *mut ListNode).write(node);
}
current.next = Some(unsafe { &mut *(addr as *mut ListNode) });
}

/// 打印当前的空闲区域
pub fn print_free_regions(&self) {
    let mut current = &self.head;
    println!("Free regions:");
    while let Some(region) = current.next.as_ref() {
        println!(
            "  Address: {:#x}, Size: {}",
            region.start_addr(),
            region.size
        );
        current = region;
    }
}

/// 计算大小和对齐
fn size_align(layout: Layout) -> (usize, usize) {

```

```

        let layout = layout
            .align_to(mem::align_of::<ListNode>())
            .expect("adjusting alignment failed")
            .pad_to_align();
        let size = layout.size().max(mem::size_of::<ListNode>());
        (size, layout.align())
    }
}

```

- 支持碎片合并，减少碎片化。

实现全局分配器 GlobalAlloc 的 alloc 和 dealloc:

```

unsafe impl GlobalAlloc for Locked<FirstFitAllocator> {
    unsafe fn alloc(&self, layout: Layout) -> *mut u8 {
        let (size, align) = FirstFitAllocator::size_align(layout);
        let mut allocator = self.lock();

        if let Some((region, alloc_start)) =
            allocator.find_region(size, align) {
            let alloc_end = alloc_start + size;
            let excess_size = region.end_addr() - alloc_end;
            if excess_size > 0 {
                allocator.add_free_region(alloc_end, excess_size);
            }
            alloc_start as *mut u8
        } else {
            ptr::null_mut()
        }
    }

    unsafe fn dealloc(&self, ptr: *mut u8, layout: Layout) {
        let (size, _) = FirstFitAllocator::size_align(layout);
        self.lock().add_free_region(ptr as usize, size)
    }
}

```

4.3 实验测试方法

- 初始化测试:初始化堆空间，打印链表结构，确保空闲区域正确。
- 内存分配测试:分配多个大小不同的内存块，验证是否按照首次适应逻辑分配。
- 内存回收测试:回收不同的内存块，观察链表是否正确更新。
- 碎片合并测试:回收相邻内存块，验证碎片是否合并。

```

fn kernel_main(boot_info: &'static BootInfo) -> ! {
    use alloc::boxed::Box;

```

```

use alloc::vec::Vec;
use blog_os::allocator::{ALLOCATOR, HEAP_SIZE, HEAP_START};
use blog_os::memory::{self, BootInfoFrameAllocator};
use x86_64::VirtAddr;

println!("Hello World{}", "!");
blog_os::init();

let phys_mem_offset =
VirtAddr::new(boot_info.physical_memory_offset);
let mut mapper = unsafe { memory::init(phys_mem_offset) };
let mut frame_allocator = unsafe
{ BootInfoFrameAllocator::init(&boot_info.memory_map) };

allocator::init_heap(&mut mapper, &mut
frame_allocator).expect("heap initialization failed");

// 打印初始空闲区域
unsafe {
    ALLOCATOR.lock().print_free_regions();
}

println!("\nTesting memory allocation and merging...");

// 分配多个内存块
let ptr1 = unsafe { ALLOCATOR.alloc(Layout::from_size_align(1024,
8).unwrap()) };
let ptr2 = unsafe { ALLOCATOR.alloc(Layout::from_size_align(1024,
8).unwrap()) };
let ptr3 = unsafe { ALLOCATOR.alloc(Layout::from_size_align(1024,
8).unwrap()) };
let ptr4 = unsafe { ALLOCATOR.alloc(Layout::from_size_align(1024,
8).unwrap()) };
let ptr5 = unsafe { ALLOCATOR.alloc(Layout::from_size_align(1024,
8).unwrap()) };
let ptr6 = unsafe { ALLOCATOR.alloc(Layout::from_size_align(1024,
8).unwrap()) };

println!("\nAfter multiple allocations:");
unsafe {
    ALLOCATOR.lock().print_free_regions();
}

```

```

// 释放不连续的内存块
unsafe {
    ALLOCATOR.dealloc(ptr2,      Layout::from_size_align(1024,
8).unwrap());
    ALLOCATOR.dealloc(ptr4,      Layout::from_size_align(1024,
8).unwrap());
    ALLOCATOR.dealloc(ptr5,      Layout::from_size_align(1024,
8).unwrap());
}

println!("\nAfter freeing some blocks:");
unsafe {
    ALLOCATOR.lock().print_free_regions();
}

let ptr7 = unsafe { ALLOCATOR.alloc(Layout::from_size_align(512,
8).unwrap()) };

println!("\nAfter multiple allocations:");
unsafe {
    ALLOCATOR.lock().print_free_regions();
}

// 释放中间的内存块，触发合并
unsafe {
    ALLOCATOR.dealloc(ptr1,      Layout::from_size_align(1024,
8).unwrap());
    ALLOCATOR.dealloc(ptr3,      Layout::from_size_align(1024,
8).unwrap());
    ALLOCATOR.dealloc(ptr6,      Layout::from_size_align(1024,
8).unwrap());
    ALLOCATOR.dealloc(ptr7,      Layout::from_size_align(512,
8).unwrap());
}

println!("\nAfter freeing middle block (should merge:");
unsafe {
    ALLOCATOR.lock().print_free_regions();
}

#[cfg(test)]
test_main();

//let mut executor = Executor::new();

```

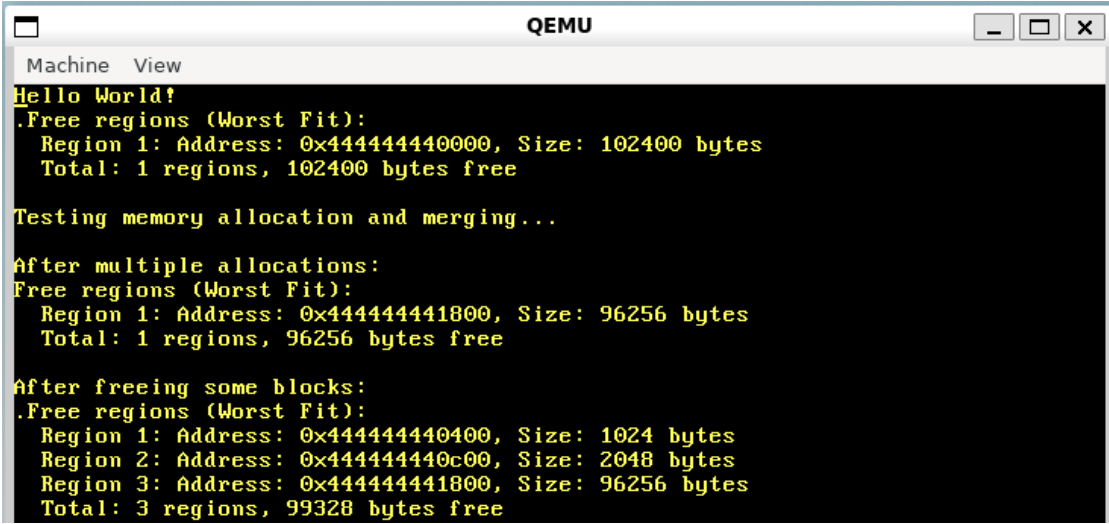


```

        //executor.spawn(Task::new(example_task()));
        //executor.spawn(Task::new(keyboard::print_keypresses()));
        //executor.run();
        loop{
        }
    }
}

```

4.4 运行结果



```

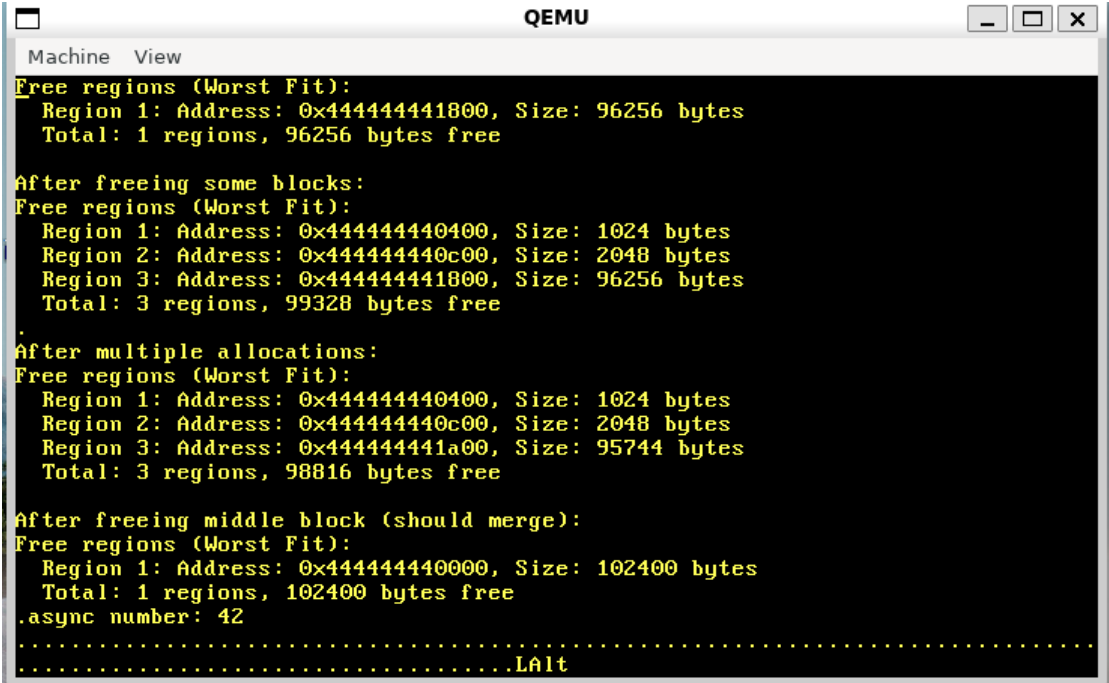
Machine View
Hello World!
Free regions (Worst Fit):
  Region 1: Address: 0x444444440000, Size: 102400 bytes
Total: 1 regions, 102400 bytes free

Testing memory allocation and merging...

After multiple allocations:
Free regions (Worst Fit):
  Region 1: Address: 0x444444441800, Size: 96256 bytes
Total: 1 regions, 96256 bytes free

After freeing some blocks:
Free regions (Worst Fit):
  Region 1: Address: 0x444444440400, Size: 1024 bytes
  Region 2: Address: 0x444444440c00, Size: 2048 bytes
  Region 3: Address: 0x444444441800, Size: 96256 bytes
Total: 3 regions, 99328 bytes free

```



```

Machine View
Free regions (Worst Fit):
  Region 1: Address: 0x444444441800, Size: 96256 bytes
Total: 1 regions, 96256 bytes free

After freeing some blocks:
Free regions (Worst Fit):
  Region 1: Address: 0x444444440400, Size: 1024 bytes
  Region 2: Address: 0x444444440c00, Size: 2048 bytes
  Region 3: Address: 0x444444441800, Size: 96256 bytes
Total: 3 regions, 99328 bytes free

After multiple allocations:
Free regions (Worst Fit):
  Region 1: Address: 0x444444440400, Size: 1024 bytes
  Region 2: Address: 0x444444440c00, Size: 2048 bytes
  Region 3: Address: 0x444444441a00, Size: 95744 bytes
Total: 3 regions, 98816 bytes free

After freeing middle block (should merge):
Free regions (Worst Fit):
  Region 1: Address: 0x444444440000, Size: 102400 bytes
Total: 1 regions, 102400 bytes free
.async number: 42
.....LAlt

```

可以看到实验中分配和回收后内存的分配情况。

五、与现有实验相比的创新点

5.1 完善性

实现了完整的首次适应内存分配算法，包括内存分配、释放、合并和打印空闲块链表状态等功能。提供了详细的代码实现指导和测试方法，方便学生理解和验证算法的正确性。

5.2 创新点

- 结合 Rust 语言的特性，实现了线程安全的自定义内存分配器。
- 提供了与全局分配器接口的集成，使得自定义内存分配器可以在 Rust 程序中使用。
- 通过实验指导书的形式，系统地介绍了首次适应算法的原理、实现和测试方法，为学生提供了全面的学习资料。